A MINIMAL FIELD HANDBOOK

The Minimal Statistics Handbook

Summaries, intervals, tests and honest comparison - the math behind every experiment.

Mosafer

scipy · statsmodels · seeds

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **2**
- 1.1 The toolchain . . . 2
- 1.2 The vocabulary . . . 2
- 1.3 The standard error of the mean . . . 2

2 Summarizing Data **4**
- 2.1 Location and spread . . . 4
- 2.2 Distributions worth knowing . . . 4
- 2.3 The central limit theorem . . . 4

3 Confidence Intervals **6**
- 3.1 The normal-interval workhorse . . . 6
- 3.2 Exact intervals for benchmarks . . . 6
- 3.3 Bootstrap: the universal interval . . . 6

4 Hypothesis Tests **8**
- 4.1 The vocabulary . . . 8
- 4.2 The t-tests . . . 8
- 4.3 Reading p-values like an adult . . . 8
- 4.4 Non-parametric alternatives . . . 9

5 Comparisons, Power, and Effect Size **10**
- 5.1 Effect size: the number that matters . . . 10
- 5.2 How many seeds? Power tells you . . . 10
- 5.3 Many comparisons: the correction . . . 10
- 5.4 The comparison checklist . . . 11

6 Regression **12**
- 6.1 Linear regression with statsmodels . . . 12
- 6.2 Reading the table . . . 12
- 6.3 Logistic regression . . . 12

7 A Bayesian Glimpse **14**
- 7.1 The mechanism . . . 14
- 7.2 Worked example: is the model above 90%? . . . 14
- 7.3 A vs. B, the Bayesian way . . . 14

8 Statistics of ML Benchmarks **16**
- 8.1 Two axes of uncertainty . . . 16
- 8.2 Comparing two models on the same test set . . . 16
- 8.3 Seeds as the unit of analysis . . . 16

A Cheat Sheet **18**
- A.1 Summaries . . . 18
- A.2 Tests . . . 18
- A.3 Regression and Bayes . . . 18
- A.4 Formulas worth memorizing . . . 19
- A.5 The reporting template . . . 19

CHAPTER 1

Orientation

Statistics is the discipline of honest conclusions under noise. Every number in a research log—an accuracy, a loss curve, a benchmark delta—is a random variable; this handbook covers the smallest working set for reasoning about them: summaries, intervals, tests, regression, and the statistics specific to ML benchmarks.

1.1 The toolchain

```
pip install numpy scipy pandas statsmodels matplotlib
```

`scipy.stats` carries the distributions and tests; `statsmodels` the regression tables; `pandas` the data handling. Everything in this book runs on a laptop in milliseconds.

1.2 The vocabulary

Term	Meaning
population vs. sample	the truth vs. what you measured
μ, σ	population mean/std — unknown, estimated
$\bar{x}$, s	sample mean/std — computed, noisy
statistic	any number computed from a sample
sampling distribution	how a statistic would vary over hypothetical re-runs of the experiment
standard error (SE)	the std of a statistic's sampling distribution — the uncertainty of an estimate

The central object is the *sampling distribution*: your $\bar{x} = 93.1\%$ is one draw from a distribution of accuracies you would get re-running the whole experiment. SE is its spread, and every tool in this book—intervals, tests, power—is arithmetic on it.

1.3 The standard error of the mean

The one formula to internalize:

$$\mathrm{SE}(\bar{x}) = \frac{s}{\sqrt{n}}$$

Quadrupling the data halves the uncertainty. This is why five seeds gives you $\sqrt{5} \approx 2.2$x tighter claims than one seed, and why `n=3` experiments cannot resolve one-point differences:

```python
import numpy as np

accs = np.array([0.928, 0.934, 0.931, 0.936, 0.927])   # 5 seeds
m, s = accs.mean(), accs.std(ddof=1)
se = s / np.sqrt(len(accs))
print(f"mean {m:.3f}  std {s:.3f}  SE {se:.4f}")
```

PITFALL

`std` of the seeds and `SE` of the mean are different numbers for different questions: std describes the *spread of runs* (which seed to expect next); SE describes the *uncertainty of the average* (what you can claim). Both belong in a paper, labeled. The formula `ddof=1` (sample std, Bessel's correction) is the one you want for 5 data points.

HOW TO READ THIS BOOK

Chapters 2–3 build summaries and intervals; Chapter 4 tests; Chapter 5 the comparison discipline; Chapter 6 regression; Chapter 7 a Bayesian glimpse; Chapter 8 the ML-benchmark specifics. Every chapter ends in code you can run on your own numbers today.

CHAPTER 2

Summarizing Data

A summary is a claim about where the data lives. Choose it to match the shape of the data, report its uncertainty, and plot the raw distribution—always.

2.1 Location and spread

Statistic	Use
mean $\bar{x}$	default for symmetric data; sensitive to outliers
median	skewed data (losses, latencies) — robust to tails
quantiles (25/50/75%)	the whole shape in three numbers
variance s^2, std s	spread in squared/original units
IQR	robust spread: 75th minus 25th percentile

```
import numpy as np
accs = np.array([0.928, 0.934, 0.931, 0.936, 0.927, 0.862])  # one bad seed

print(np.mean(accs), np.median(accs))     # 0.920 vs 0.930 - outlier visible
print(np.quantile(accs, [0.25, 0.5, 0.75]))
```

The lone 0.862 seed drags the mean by a point while the median barely moves. Neither is "right": report the one that matches the question, and show the distribution so the reader sees the outlier too.

2.2 Distributions worth knowing

Distribution	Models	Key facts
Normal $\mathcal{N}(\mu, \sigma^2)$	aggregated noise, accuracies over seeds	68/95/99.7% within $1\sigma/2\sigma/3\sigma$
Binomial(n, p)	correct/incorrect counts over n test items	mean np, variance $np(1-p)$
Bernoulli(p)	one binary outcome	the $n = 1$ binomial
Uniform	ignorance between bounds	the usual noninformative prior (Ch. 7)

```
from scipy import stats

stats.norm.cdf(1.96)                    # 0.975 - the famous constant
stats.norm.ppf(0.975)                   # 1.9599... - and back
stats.binom.pmf(k=52, n=100, p=0.5)     # exactly 52 correct by chance
stats.binom.cdf(52, 100, 0.5)           # P(at most 52 correct)
```

2.3 The central limit theorem

Sum or average enough independent contributions and the result is approximately normal *whatever the parts looked like*. This is why the normal shows up everywhere, and why the SE-based intervals of Chapter 3 work on accuracies, losses, and latencies alike:

```python
import numpy as np
rng = np.random.default_rng(0)
means = [rng.exponential(2.0, 1000).mean() for _ in range(20000)]
print(np.std(means))      # SE of the mean of 1000 exp(2) draws
                          # ~= 2/sqrt(1000) - normal, though exp is skewed
```

PITFALL

Averaging is where skew hides. Mean training loss over a noisy run can be dominated by spikes; mean latency by cold starts. When the distribution is skewed, report the median (and IQR) or the mean of the clipped distribution, and say which. A mean without its distribution is a rumor.

PLOT FIRST, COMPUTE SECOND

Before any test or interval: histogram or strip-plot the data. Every classic statistical failure—outliers, bimodality, a saturated accuracy at 1.0, the lone 0.862 seed—is visible in five seconds of plotting and invisible in five minutes of test statistics.

CHAPTER 3

Confidence Intervals

A point estimate is a claim without a hedge. A confidence interval quantifies the hedge: given the data, plausible values of the truth.

3.1 The normal-interval workhorse

For a sample mean with n reasonably large:

$$\bar{x} \pm z_{0.975} \cdot \frac{s}{\sqrt{n}} \quad \approx \quad \bar{x} \pm 1.96 \cdot \mathrm{SE}$$

Reading: if the experiment were re-run many times, 95% of such intervals would contain the true mean. The interval is about the *procedure*, not this one interval:

```
import numpy as np
from scipy import stats

accs = np.array([0.928, 0.934, 0.931, 0.936, 0.927])
m, se = accs.mean(), accs.std(ddof=1) / np.sqrt(len(accs))
z = stats.norm.ppf(0.975)
print(f"95% CI: {m:.3f} +- {z*se:.3f}")        # 0.931 +- 0.004

t = stats.t.ppf(0.975, df=len(accs)-1)         # small n: use t, not z
print(f"95% CI: {m:.3f} +- {t*se:.3f}")        # wider: 0.931 +- 0.006
```

With $n = 5$, the Student-t multiplier (2.78) is noticeably larger than 1.96 — the interval admits what small samples cannot know.

3.2 Exact intervals for benchmarks

An accuracy on n test items is a binomial proportion — exact intervals need no normal approximation at all:

```
from statsmodels.stats.proportion import proportion_confint

lo, hi = proportion_confint(count=931, nobs=1000, alpha=0.05,
                            method="beta")     # Clopper-Pearson
print(f"95% CI: [{lo:.3f}, {hi:.3f}]")         # [0.914, 0.945]
```

A 93.1% accuracy on 1000 items honestly reads `[0.914, 0.945]` — test-set size is part of the claim, and 500-item test sets give intervals wide enough to swallow most leaderboard deltas.

3.3 Bootstrap: the universal interval

When the statistic is exotic (median F1, difference of AUCs, a ratio) or the distribution is ugly, resample:

```python
import numpy as np

def ci_bootstrap(x, stat, B=10000, alpha=0.05, rng=None):
    rng = rng or np.random.default_rng(0)
    x = np.asarray(x)
    idx = rng.integers(0, len(x), size=(B, len(x)))
    stats_ = stat(x[idx], axis=1)
    lo, hi = np.quantile(stats_, [alpha/2, 1 - alpha/2])
    return stat(x), lo, hi

accs = np.array([0.928, 0.934, 0.931, 0.936, 0.927])
print(ci_bootstrap(accs, np.median))
```

Resample with replacement B times, recompute the statistic, take percentiles. It replaces formulas with compute and works for *any* statistic — the default answer whenever no textbook interval exists.

PITFALL

Three interval misreadings: (1) "the true value is in this interval with 95% probability" — no; the *procedure* covers 95% of the time; (2) reporting the interval from ONE seed's test accuracy and calling it seed-uncertainty — item-uncertainty and seed-uncertainty are different axes (Ch. 8); (3) $n = 3$ seed intervals via the normal approximation — with tiny n, bootstrap the seeds or use t-multipliers, and say how many seeds there were.

WHICH INTERVAL WHEN

Mean over seeds, $n \geq 10$: normal or t interval on the mean. Mean over seeds, $n < 10$: t interval. Any weird statistic: percentile bootstrap. A benchmark proportion with many items: exact binomial (beta) interval. Report the method next to the interval — "mean ± std over 5 seeds" is a method statement, not decoration.

CHAPTER 4

Hypothesis Tests

A test asks: could this result be an artifact of noise? It computes the probability of data at least this extreme *assuming no effect* — the p-value — and you decide whether the noise explanation survives.

4.1 The vocabulary

Term	Meaning
H_0 (null)	"no difference" — the boring explanation
test statistic	a number measuring signal vs. noise (t, χ^2)
p-value	Pr(data this extreme | H_0) — noise plausibility, *not* Pr(H_0)
significance level α	the p-value below which you act (0.05 by convention)
type I / type II error	false alarm / missed effect
power	Pr(detect a real effect) — Ch. 5

4.2 The t-tests

```
from scipy import stats

a = np.array([0.928, 0.934, 0.931, 0.936, 0.927])   # method A, 5 seeds
b = np.array([0.941, 0.938, 0.944, 0.940, 0.943])   # method B, same seeds

# two independent groups (different seeds):
t, p = stats.ttest_ind(a, b, equal_var=False)       # Welch: don't assume equal var

# SAME seeds on both methods -> paired is more powerful:
t, p = stats.ttest_rel(b, a)
print(f"paired t={t:.2f}, p={p:.5f}, mean delta {np.mean(b-a):+.4f}")
```

Paired tests compare *within-seed differences*: the shared luck of each seed cancels, so the test sees the treatment effect, not the seed lottery. If your experiment uses the same seeds for both methods, `ttest_rel` is the right test and `ttest_ind` wastes the design.

4.3 Reading p-values like an adult

- $p = 0.03$ does not mean "95% chance B beats A". It means: under no-effect, data this extreme happens 3% of the time.
- Non-significance is not evidence of equality — absence of evidence is not evidence of absence. Report the effect size with its interval: "+0.9 ± 0.4 points" says more than any p.
- With 3 seeds, skip the theater: report all runs. With 5–10, paired tests start meaning something.
- The threshold $\alpha = 0.05$ is a convention, not a law — but silently moving it after seeing the p-value is a crime.

PITFALL

Multiple comparisons quietly invalidate α: run 20 comparisons at $\alpha = 0.05$ and expect one "significant" hit by pure chance. If you compared many variants, correct — Chapter 5 does exactly this. Pre-declare the ONE comparison that matters and treat the rest as exploratory, labeled as such.

4.4 Non-parametric alternatives

Small n, weird distributions, or pure rankings:

```
u, p = stats.mannwhitneyu(b, a, alternative="two-sided")  # ranks, 2 groups
w, p = stats.wilcoxon(b, a)            # paired ranks
```

`wilcoxon` on per-seed deltas asks: are the differences consistently one-signed? With 5 seeds and a clean sign pattern it is often the more honest test than a t-test whose normality assumption you cannot check.

THE PRE-REGISTRATION INSTINCT

Decide the metric, the seeds, the baseline, and the test BEFORE seeing the results, and write them in the README. Everything chosen after looking at outcomes is a forking path; the Ch. 11 discipline of the PyTorch handbook (configs committed before runs) is pre-registration for the rest of us.

CHAPTER 5

Comparisons, Power, and Effect Size

Comparing methods is the bread and butter of research, and the bread is stale when power and multiplicity are ignored. This chapter is the comparison discipline: effect sizes first, corrections for many comparisons, and how many seeds you actually need.

5.1 Effect size: the number that matters

The question is never "is p small?" but "how big is the difference, and how sure are we?"

```
import numpy as np
from scipy import stats

d = b - a                                    # per-seed deltas
mean_d, sd_d = d.mean(), d.std(ddof=1)
cohen = mean_d / sd_d                        # Cohen's d for paired data
t, p = stats.ttest_rel(b, a)
ci = stats.t.interval(0.95, loc=mean_d, scale=stats.sem(d),
                      df=len(d)-1)
print(f"delta {mean_d*100:+.2f} pts, 95% CI [{ci[0]*100:+.2f}, {ci[1]*100:+.2f}],
     ↪ d={cohen:.2f}")
```

Rough d reading: 0.2 small, 0.5 medium, 0.8 large. In ML terms: +0.9 points with CI $[+0.5, +1.3]$ over 5 seeds is a real, small effect; the same delta with CI $[-0.2, +2.0]$ is unproven regardless of any p-value.

5.2 How many seeds? Power tells you

Power = probability of detecting an effect that exists. Flip it into a planning question: for the effect I care about, how many seeds until the test has 80% power?

```
from statsmodels.stats.power import TTestIndPower

analysis = TTestIndPower()
n = analysis.solve_power(effect_size=1.5,   # expected d (large)
                         alpha=0.05, power=0.8,
                         ratio=1.0, alternative="two-sided")
print(f"seeds per method: {np.ceil(n):.0f}")
```

Large expected effects need few seeds; subtle ones need more than most labs are willing to run — which is itself a finding: *if you cannot afford the seeds, sharpen the question or accept an interval with a wide bow.*

5.3 Many comparisons: the correction

Testing m hypotheses at $\alpha = 0.05$ inflates false alarms toward $1 - 0.95^m$ — 20 comparisons, ≈64% chance of at least one false positive. Correct when the family of tests is real:

```python
from statsmodels.stats.multitest import multipletests

pvals = np.array([0.001, 0.008, 0.04, 0.21, 0.44, 0.87])
reject, p_corr, _, _ = multipletests(pvals, alpha=0.05, method="holm")
print(p_corr)          # Holm-adjusted p-values
```

`holm` controls the false-discovery *rate of the family* and is uniformly better than plain Bonferroni; `fdr_bh` is the gentler standard when the family is exploratory screening.

PITFALL

The garden of forking paths: every extra plot examined, threshold tuned, and seed swapped is an implicit comparison that never entered the correction. The defense is procedural, not mathematical — commit the analysis plan before the runs (Git handbook Ch. 7), and report which comparisons were planned versus exploratory.

5.4 The comparison checklist

1. Same seeds, same splits, same budget — or the comparison is fiction.
2. Effect size with interval (paired deltas when the design allows).
3. Paired test for the headline claim; Holm-corrected across the family of variants.
4. n seeds chosen by a power calculation, or an honest statement of what $n = 3$ cannot show.
5. One test set, touched once — the test set is spent the moment it steers any decision.

READING A LEADERBOARD

The table that beats the table above it by 0.2 points on a 1000-item test set, with unknown seeds, is a coin toss wearing a suit. What changes your beliefs: deltas larger than the binomial interval (Ch. 3), repeated across seeds, on the same splits, with the comparison declared in advance.

CHAPTER 6

Regression

Regression models an outcome as a function of inputs — the general form of "how much does X move Y?" Every coefficient comes with a standard error and interval, which is why it doubles as the hypothesis-testing workhorse of observational data.

6.1 Linear regression with statsmodels

```
import pandas as pd
import statsmodels.formula.api as smf

df = pd.DataFrame({
    "acc":     [0.928, 0.934, 0.931, 0.936, 0.927, 0.941, 0.938, 0.944],
    "seeds":   [1, 1, 1, 1, 1, 1, 1, 1],
    "lr":      [1e-4, 1e-4, 3e-4, 3e-4, 1e-3, 1e-3, 3e-3, 3e-3],
    "width":   [64, 128, 64, 128, 64, 128, 64, 128],
    "method":  [0, 1, 0, 1, 0, 1, 0, 1],
})
m = smf.ols("acc ~ lr + width + method", data=df).fit()
print(m.summary())        # coefficients, SEs, t-stats, p-values, R^2
```

The summary table is the point: each row says how much the outcome moves per unit of the input, holding the others fixed, with its uncertainty. `m.conf_int()` gives coefficient intervals; `m.predict(new)` extrapolates (carefully).

6.2 Reading the table

Column	Meaning
`coef`	estimated effect per unit of the predictor
`std err`	uncertainty of that estimate
`[0.025, 0.975]`	95% coefficient interval — does it contain zero?
`P>\textbar t\textbar`	p-value of "this coefficient is really zero"
`R-squared`	fraction of variance explained — context, not a grade

Categorical predictors (datasets, methods) enter as `C(method)` in the formula and produce one-vs-reference coefficients — the ablation table as a regression.

6.3 Logistic regression

For binary outcomes (and as the linear layer of every classifier):

```
m = smf.logit("correct ~ method + width", data=df_items).fit()
print(m.summary())                  # log-odds coefficients
print(np.exp(m.params["method"]))   # odds ratio: interpret this
```

Coefficients are log-odds; exponentiate for odds ratios — `1.4` means 40% higher odds of a correct prediction per unit. For predictions rather than interpretation, scikit-learn's `LogisticRegression` is the operational twin.

PITFALL

Four regression classics: (1) correlated predictors (lr and batch size move together) make individual coefficients meaningless — check correlations, ablate, or drop; (2) extrapolation: a model fit on lr in $[10^{-4}, 10^{-3}]$ says nothing about 10^{-1}; (3) confound vs. cause: regression adjusts for measured confounders, not for the ones you forgot to log — which is why logging everything (Git handbook Ch. 7) pays statistically; (4) p-hacking via specification search: trying 15 formulas and reporting the best summary is the multiple-comparisons sin in a trench coat (Ch. 5).

WHEN REGRESSION BEATS AVERAGING

Averaging over seeds collapses everything else into noise; regression keeps it: 40 runs across lr, width, and dataset yield coefficient estimates *and* interactions (`acc ~ lr * width`) from data you already have. The ablation grid is a regression waiting to be fit.

CHAPTER 7

A Bayesian Glimpse

Frequentist tools ask "how surprising is the data under H_0?". Bayesian tools ask a more direct question: *given the data, what should I believe?* This chapter is the 20-minute version: one worked example, one interval type, one comparison method.

7.1 The mechanism

Start with a *prior* belief about a quantity, update it with likelihood from data, and read out the *posterior*:

$$\underbrace{p(\theta \mid \text{data})}_{\text{posterior}} \propto \underbrace{p(\text{data} \mid \theta)}_{\text{likelihood}} \cdot \underbrace{p(\theta)}_{\text{prior}}$$

The posteriors of everything you care about become the objects you report: $\Pr(\theta > 0.9 \mid \text{data})$, $\Pr(\text{B} > \text{A} \mid \text{data})$ — probabilities of *hypotheses*, which frequentist p-values explicitly are not.

7.2 Worked example: is the model above 90%?

A model scores 93 correct of 100. Beta-Binomial gives the exact posterior with a uniform prior (Beta(1, 1)):

```
from scipy import stats
import numpy as np

posterior = stats.beta(1 + 93, 1 + 7)      # uniform prior + data
print(f"posterior mean: {posterior.mean():.3f}")
print(f"95% credible interval: {posterior.ppf([0.025, 0.975]).round(3)}")
print(f"P(accuracy > 0.90 | data): {1 - posterior.cdf(0.90):.3f}")
```

The credible interval `[0.868, 0.964]` says: given the data (and the prior), the accuracy lies in this range with probability 0.95 — the sentence everyone *wants* a confidence interval to say.

7.3 A vs. B, the Bayesian way

Two models, same 100 test items. Sample each posterior and compare:

```
rng = np.random.default_rng(0)
pa = stats.beta(1 + 910, 1 + 90)    # model A: 910/1000
pb = stats.beta(1 + 925, 1 + 75)    # model B: 925/1000
sa, sb = pa.rvs(200_000, random_state=rng), pb.rvs(200_000,
                                                  random_state=rng)
print(f"P(B better): {(sb > sa).mean():.3f}")     # ~0.94
```

"Model B is better with probability 0.94 given this test set" — a direct, communicable claim. Sample sizes in the hundreds of thousands make the Monte-Carlo error negligible.

> **PITFALL**

The prior is a modeling decision, not a decoration: with 100 items a uniform prior vs. a skeptical prior (concentrated at 0.5) shifts the posterior visibly. Report the prior, run a prior-sensitivity check (two priors, two posteriors), and never tune the prior until the conclusion looks right — it is the garden of forking paths with a Bayesian coat of paint.

WHEN TO GO FURTHER

For everything in this chapter there is an exact conjugate shortcut; beyond it (hierarchical models, posterior over a whole training run), tools like PyMC or numpyro sample posteriors you cannot write down. The mindset — parameters are distributions, data updates them — transfers wholesale. Start with Kruschke's *Doing Bayesian Data Analysis* when this chapter feels too short.

CHAPTER 8

Statistics of ML Benchmarks

Everything so far applies to ML numbers; this chapter adds the benchmark-specific structure: two uncertainty axes (test items and seeds), a budget split between them, and the tests designed for per-item predictions.

8.1 Two axes of uncertainty

Axis	Question it answers	Tool
test items	how reliable is ONE run's accuracy?	binomial / bootstrap over items (Ch. 3)
seeds	how reliable is the training procedure?	mean ± std over seeds, paired tests (Ch. 4)

They are independent and both real: `[0.914, 0.945]` from 1000 test items says nothing about what the fifth seed would have scored, and five seeds say nothing about the 500-item test set's width. When budget is tight, more test items is usually the better buy — item-width shrinks as $1/\sqrt{n}$ and it is the axis reviewers probe first.

8.2 Comparing two models on the same test set

Per-item predictions are paired (same questions), which is information the naive two-proportion test throws away. McNemar's test uses exactly the discordant pairs:

```
from statsmodels.stats.contingency_tables import mcnemar

# both models on the SAME 1000 items:
b00, b01 = 880, 30      # both wrong; A wrong, B right
b10, b11 = 15, 75       # A right, B wrong; both right
table = [[b00, b01], [b10, b11]]
res = mcnemar(table, exact=False, correction=True)
print(res.statistic, res.pvalue)   # built from the 30 vs 15 discordants
```

Only the 45 items the models disagree on carry the signal; the 955 agreements are theater. Bootstrap alternative: resample test items, recompute both accuracies, take the percentile interval of the *difference* — pairs preserved:

```
idx = rng.integers(0, n, size=(B, n))
diffs = acc_b[idx].mean(axis=1) - acc_a[idx].mean(axis=1)
lo, hi = np.quantile(diffs, [0.025, 0.975])
```

8.3 Seeds as the unit of analysis

For the training procedure (not one run), the seed is the experimental unit and Ch. 5's rules apply verbatim: paired t / Wilcoxon on per-seed deltas, effect size with interval, Holm across the family of baselines. Five seeds is the honest floor; three is an anecdote with a standard error.

PITFALL

The test-set-as-training-set sin is statistical before it is ethical: every hyperparameter picked while watching test scores fits the test set — the multiple-comparisons problem with unbounded m. A test set consulted 50 times is a validation set wearing a crown. Hold it out, touch it once, and put the real selection loop on validation splits.

THE BENCHMARK REPORT, ASSEMBLED

For each method: accuracy with its binomial interval (items), mean ± std over seeds (procedures), paired delta vs. the baseline with bootstrap interval (the claim), number of seeds and test size, and which comparisons were pre-declared. One table, five numbers, zero hand-waving — this is what "state of the art" should mean.

APPENDIX A

Cheat Sheet

A.1 Summaries

Task	Code
Moments	`x.mean()`, `x.std(ddof=1)`, `np.quantile(x, [0.25, 0.5, 0.75])`
SE of mean	`x.std(ddof=1) / np.sqrt(len(x))`
95% CI (normal)	`m + stats.norm.ppf(0.975) * se`
95% CI (small n)	`stats.t.interval(0.95, len(x)-1, loc=m, scale=stats.sem(x))`
Exact proportion	`proportion_confint(k, n, method="beta")`
Bootstrap	`ci_bootstrap(x, np.median)` (Ch. 3)

A.2 Tests

Task	Code
Two groups, independent	`stats.ttest_ind(a, b, equal_var=False)`
Two groups, paired	`stats.ttest_rel(b, a)`
Ranks, two groups	`stats.mannwhitneyu(b, a)`
Ranks, paired	`stats.wilcoxon(b, a)`
Proportions, paired items	`mcnemar([[b00, b01], [b10, b11]])`
Many comparisons	`multipletests(pvals, method="holm")`
Power \(\to \) n	`TTestIndPower().solve_power(effect_size=d, alpha=0.05, power=0.8)`

A.3 Regression and Bayes

Task	Code
Linear	`smf.ols("acc ~ lr + C(method)", data=df).fit()`, `.summary()`
Logistic	`smf.logit("correct ~ method", data=df).fit()`; odds ratio `np.exp(coef)`
Beta posterior	`stats.beta(1 + k, 1 + n - k)`
Credible interval	`posterior.ppf([0.025, 0.975])`
Pr($\theta > c$)	`1 - posterior.cdf(c)`
P(B > A)	`(pb.rvs(B) > pa.rvs(B)).mean()`

A.4 Formulas worth memorizing

Quantity	Formula
SE of the mean	$s/\sqrt{n}$
95% normal interval	$\bar{x} \pm 1.96 \cdot \mathrm{SE}$
Cohen's d (paired)	$\bar{d}/s_d$
Binomial std (accuracy)	$\sqrt{p(1-p)/n}$
Bonferroni	$\alpha_{\mathrm{per}} = \alpha/m$

A.5 The reporting template

1. Distribution of the raw numbers (plot, always).
2. Point estimate with interval, method and n stated ("mean ± std over 5 seeds").
3. Effect size for the comparison, with paired interval.
4. Test for the headline claim (paired, corrected across the family), declared in advance.
5. Test-set size and seeds — the two axes of the claim.

> **THE ONE HABIT**
>
> Plot the raw data before any statistic; report intervals next to every point estimate; declare the comparison before running it. Statistics is not a gate after the experiment — it is the design of the experiment, and it starts at the config file.